

SEMANA 3 ACADEMIA MONTERREY

- Introducción a Programación Reactiva

El tema de la programación reactiva se tomó el día 4 de la semana 3 de la academia.

La programación reactiva es un estilo de programación en el que se trabaja con flujos de datos que llegan de forma asíncrona, en lugar de esperar a que finalicen las operaciones. Esto quiere decir que las llamadas en el programa se realizan y continúan procesando mientras llega un resultado, en lugar de esperar por completo a que un programa específico responda; los hilos no se quedan esperando.

El punto más importante entre la programación reactiva y la tradicional, también conocida como programación por bloques, es que en la programación tradicional, la ejecución bloquea hilos hasta que se obtenga una respuesta del servicio llamado, lo que resulta en un sistema costoso si necesitan crearse diversas peticiones en el mismo programa. En la programación reactiva, los hilos se liberan y resumen al obtener una respuesta, no se bloquean, lo que permite poder manejar varias peticiones a la vez.

Spring WebFlux utiliza la librería Project Reactor, el cual contiene dos tipos de datos primarios para representar streams asíncronos de datos. Estos son: Mono<T> Flux<T>.

- **Mono**

En nuestro programa tenemos:

```
@GetMapping("/employees/{id}")
public Mono<Employee> findById(@PathVariable int
id) {
    log.info("-> pediste el empleado {} (hilo: {})",
id, Thread.currentThread().getName());

    return repo.findById(id)
        .doOnNext(e -> log.info("<- llego {}
(hilo: {})",
```

```
        e.firstName(),  
Thread.currentThread().getName())  
        .switchIfEmpty(Mono.error(new  
ResponseStatusException(  
        HttpStatus.NOT_FOUND, "No  
existe el empleado " + id)));  
}
```

Un `Mono<T>` puede representar valores en cero, o un valor, o un error, pero nunca más de un solo valor. En este caso, al obtener un nombre de un empleado, solo buscamos una sola respuesta. El framework espera automáticamente a que el `Mono` finalice y envía el resultado.

Si utilizáramos Spring MVC, es decir, la versión bloqueante, la ejecución tomaría mucho tiempo si se buscaran varios hilos para responder a la solicitud. En WebFlux, la versión reactiva, el hilo de búsqueda no tiene que terminar el trabajo para que pueda realizarse otra petición, simplemente espera a que se regrese un resultado, el cual, como se mencionó anteriormente, puede ser un empleado, un error, o ninguno.

- **Flux**

En nuestro programa tenemos:

```
@GetMapping(produces =  
MediaType.APPLICATION_JSON_VALUE)  
public Flux<Lectura> comoJson() {  
    log.info("[JSON] el cliente esperara a que  
terminen las 5 lecturas");  
    return sensor.lecturas().take(5);  
}
```

Y

```
@GetMapping(path = "/stream", produces =
MediaType.TEXT_EVENT_STREAM_VALUE)
public Flux<Lectura> comoStream() {
    log.info("[STREAM] el cliente recibira una
lectura por segundo");
    return sensor.lecturas().take(20);
}
```

En estos ejemplos, nuestro endpoint regresa un flujo de respuestas en lugar de solo una. El programa regresa un flujo de lecturas de temperatura en un evento a la vez, como por ejemplo:

data: {"temp":21.1}

data: {"temp":21.3}

data: {"temp":21.2}

Y podemos establecer que regrese nuestros resultados en formato JSON o SSE, el cual envía un flujo en vivo de la ejecución del programa. Esto es útil en otros contextos, como por ejemplo localizaciones GPS, mensajes de chat, precios del mercado de activos, entre otros. En nuestro ejemplo específico, recibiremos un flujo de temperaturas constantemente (un total de 5 o 20, dependiendo de cual sea ejecutado), así podemos limitar nuestros resultados en lugar de obtener un flujo infinito de lecturas en nuestra ejecución.